

Informe — Trabajo Práctico Integrador

Mejora integradora: Reporte de Rotación de Inventario
FerreRAP — Sistema de Gestión de Stock
Ingeniería de Software II · UCP · 1er cuatrimestre 2026

Equipo nipintucu

Integrante	Rol
Santiago Gonzalez	Scrum Master
Luciano Fohrholtz	Dev Lead
Juan Carlos Abente	QA Lead
Emilia Auras	UX Lead

1. Mejora elegida

Se implementó un **Reporte de Rotación de Inventario** como mejora integradora principal de FerreRAP.

¿Qué problema resuelve?

El sistema registraba cada movimiento de stock (entradas y salidas), pero esa información **nunca se analizaba**. No había forma de saber qué productos rotan rápido, cuáles están estancados, ni cuántos días faltan para que un producto se agote. Los dos reportes que ya existían (stock actual y productos a reponer) son fotografías estáticas del inventario; faltaba una mirada sobre el **movimiento** real de las ventas.

¿Por qué es una mejora respecto al estado anterior (TP2)?

- Reutiliza el patrón de diseño **Strategy** que el sistema ya tenía: se agrega una estrategia nueva (ReporteRotacion) sin modificar el contexto GeneradorReporte ni las estrategias existentes. Esto demuestra la **extensibilidad** del diseño: evolucionar el sistema sin romperlo.
- Convierte datos que estaban subutilizados (la tabla de movimientos) en información útil para la toma de decisiones de reposición.
- Genera, como subproducto, el archivo de datos numéricos que exige la integración con Estadística (ver sección 3).

2. Implementación

La mejora se implementó en tres capas, respetando la arquitectura existente (Flask + Supabase + frontend HTML/JS):

a) Capa de dominio — nueva estrategia (models.py)

Se creó la clase ReporteRotacion(EstrategiaReporte), que implementa la misma interfaz que las estrategias previas. A diferencia de ellas, recibe el historial de movimientos en su constructor; el método generar(productos) conserva la misma firma, por lo que **GeneradorReporte no requirió ningún cambio**. La estrategia agrupa las salidas por producto y calcula, para cada uno: total de unidades vendidas, promedio de unidades por movimiento de salida y días estimados hasta el quiebre de stock.

b) Capa de API — nuevos endpoints (app.py)

- GET /api/reportes/rotacion — devuelve el reporte en JSON, construyendo el contexto con la estrategia de rotación.
- GET /api/reportes/rotacion/csv — exporta el mismo cálculo como archivo CSV para la materia Estadística.

c) Capa de presentación — frontend (index.html)

Se agregó el botón “**Rotación de inventario**” en el selector de reportes, una función que consume el endpoint y muestra los resultados en la tabla con encabezados propios, y un enlace de descarga directa del CSV.

Fraagmento representativo (cálculo del indicador clave):

```
dias_hasta_quiebre = int(stock_actual / promedio_salida) # si promedio > 0
```

Aclaración de diseño: el indicador “días hasta quiebre” es una **estimación simplificada**. Usa el promedio de unidades por movimiento de salida como proxy del consumo, ya que no se dispone de un rango temporal exacto entre movimientos. Es una aproximación válida para el alcance del trabajo y puede refinarse a futuro incorporando las fechas reales de cada movimiento.

3. Integración con Estadística

El endpoint de exportación genera un archivo **CSV** con columnas **numéricas reales**, aptas para análisis estadístico genuino (medias, desvíos, correlaciones; por ejemplo, correlación entre precio y rotación). El archivo se llama `ferrerap_rotacion_AAAAMMDD_HHMM.csv` y contiene las columnas: producto, categoria, stock_actual, total_salidas, promedio_salida, dias_hasta_quiebre, precio_costo, precio_venta.

Eiemplo — primeras 5 líneas del archivo generado:

```
producto,categoria,stock_actual,total_salidas,promedio_salida,dias_hasta_quiebre,precio_costo,pr
ecio_venta
Cable 2.5mm x mt,Electricidad,22,189,4.61,4,233.0,350.0
Destornillador Ph,Herramientas,15,45,2.05,7,530.0,795.0
Martillo 500g,Herramientas,2,13,1.18,1,1000.0,1500.0
Tornillos 4x40,Fijaciones,48,10,2.5,19,430.0,650.0
Latex blanco 4L,Pinturas,9,3,1.0,9,2530.0,3800.0
```

Cada fila es una observación y cada columna numérica una variable lista para procesar en la materia Estadística, sin necesidad de limpieza previa.

4. Verificación

El cálculo de la estrategia se valida con una **prueba unitaria** que usa datos conocidos y comprueba que los resultados coincidan con los esperados, de forma independiente de la base de datos:

- Se construye una lista de productos y de movimientos de prueba con valores fijos.
- Se ejecuta `ReporteRotacion(movimientos).generar(productos)`.
- Se verifica que `total_salidas` sume correctamente, que `promedio_salida` sea el promedio por evento, y que `dias_hasta_quiebre` dé el entero esperado.
- Caso borde: un producto sin movimientos devuelve `total_salidas = 0` y `dias_hasta_quiebre` nulo.

La prueba está en `tests/test_rotacion.py` y se ejecuta con `pytest tests/test_rotacion.py -v`. Los 5 casos pasan correctamente. Además, la funcionalidad se comprobó manualmente sobre la aplicación desplegada en Vercel, generando el reporte y descargando el CSV.

5. Reflexión del equipo

¿Qué nos costó más?

La mayor dificultad no estuvo en programar la mejora en sí, sino en **coordinar el trabajo en equipo sobre el mismo repositorio**. Al trabajar varios integrantes en paralelo nos pasó que un rediseño completo de la interfaz se subió mientras otra parte del equipo tenía cambios sin integrar, lo que nos obligó a reorganizar el repositorio, partir de una copia limpia y reaplicar las mejoras sobre la versión actualizada. Eso nos enseñó a la fuerza la importancia de trabajar con **ramas separadas**, hacer `git pull` antes de empezar y comunicar qué estaba tocando cada uno. También nos llevó tiempo la **configuración del entorno** (variables de entorno de Supabase, despliegue en Vercel) y entender cómo extender el patrón Strategy sin romper el código existente.

¿Qué aprendimos?

Aprendimos el valor concreto de los **patrones de diseño**: gracias a que el sistema ya usaba Strategy, pudimos agregar un reporte completamente nuevo sin tocar una sola línea del código que ya funcionaba, lo que nos dio seguridad de no romper nada. Entendimos también que una sola funcionalidad bien pensada puede cubrir **dos**

objetivos a la vez (mejorar el producto y, al mismo tiempo, producir el insumo de datos para Estadística). Por último, reforzamos prácticas de ingeniería de software que antes veíamos como teoría: versionar con Git de forma ordenada, escribir una prueba automatizada que verifique el cálculo, y documentar los cambios para que cualquiera del equipo pueda explicarlos.

Repositorio: github.com/LuchoFohrholtz/is2-2026-nipintucu · Aplicación desplegada en Vercel · FerreRAP · Equipo nipintucu · IS2 · UCP · 2026